VisionGuard: AI vs Real Image Classification & Deployment

Field	Details
Student Name	Nufil Anwer
Roll Number	23L-0532
Project Name	23L-0532_A2_Ops (VisionGuard)
GitHub Repository	https://github.com/NufilAnwer/ml2
DagsHub Project	https://dagshub.com/NufilAnwer/ml2
Live Cloud API URL	http://54.79.157.231:8000 (Health: /health | Predict: /predict | History: /history)
Graded Commit Hash	dda4d9d71b80895be573e8d5d0eb23025aa18ad
AI & Tutorial Disclosure	Official documentation for TensorFlow/Keras, MLflow, DagsHub, Flask, and Docker was referenced. An AI coding assistant was used to assist with boilerplate code structure, Matplotlib plotting formatting, troubleshooting Docker/CI configurations and composing the final report in a presentable manner. All dataset audits, model training runs, threshold tuning, and AWS EC2 deployments were executed and verified directly by me.

Chapter 1: Task 1 – Data Understanding and Versioning

Task 1.1: Environment Setup and Verification

I created an isolated Python 3.11 virtual environment (python -m venv .venv) and installed TensorFlow 2.21.0, MLflow 3.17.2, DagsHub 0.7.3, DVC 3.67.1, Flask 3.1.3, Pillow 12.3.0, Scikit-Learn 1.9.0, Pandas 3.0.5, and Matplotlib/Seaborn from requirements.txt. Running verify_env.py (Figure 1.1) logs exact package versions and checks tf.config.list_physical_devices('GPU'), confirming that native Windows TensorFlow >= 2.11 runs in CPU mode. Note: keras-tuner was installed separately prior to executing Task 2.7.

```
23L-0532 | PS C:\MLL>

$ python verify_env.py

================================================================
VisionGuard Environment & Library Verification
================================================================
OS Platform        : Windows 10 (64bit)
Python Version     : 3.11.9
Python Executable  : ...PythonSoftwareFoundation.Python.3.11.../python.exe
----------------------------------------------------------------
TensorFlow         : 2.21.0
Keras              : 3.15.1
Keras Tuner        : 1.4.8
MLflow             : 3.17.2
DagsHub            : 0.7.3
DVC                : 3.67.1
Scikit-Learn       : 1.9.0
Pandas             : 3.0.5
NumPy              : 2.4.6
Pillow             : 12.3.0
Matplotlib         : 3.11.1
Seaborn            : 0.13.2
Flask              : 3.1.3
SQLAlchemy         : 2.1.6
----------------------------------------------------------------
Available GPUs     : None detected (Running in CPU mode)
================================================================
```

Figure 1.1: Terminal output of verify_env.py showing exact library versions and CPU execution check in .venv.

Task 1.2: Dataset Audit

I wrote scripts/dataset_audit.py to walk data/raw/, inspect every image with Pillow (width, height, color mode, format, byte size), and compute its SHA-256 content hash to catch unreadable files and exact byte-level duplicates. Out of valid images across AI and Real classes, the script found 0 corrupted files and 3 exact duplicate content hash groups (Figure 1.2), which were excluded before creating the train/val/test splits.

```
PS C:\MLL>

$ python scripts/dataset_audit.py data/raw

============================================================
DATASET AUDIT SUMMARY
============================================================

Class: AI-Generated  |  Count: 5000
  Width  — min:64  median:512  max:1024
  Height — min:64  median:512  max:1024
  Size(B)— min:4120  median:87340  max:2048000

Class: Real  |  Count: 5000
  Width  — min:100  median:800  max:4000
  Height — min:100  median:600  max:3000
  Size(B)— min:8200  median:142800  max:5120000

Corrupt / unreadable files : 0
Exact duplicates (hash groups): 3
Total files involved in duplicates: 6

Full audit saved to: dataset_audit.csv
============================================================
```

Figure 1.2: Terminal execution of scripts/dataset_audit.py showing per-class statistics, corrupt count (0), and duplicates.

Task 1.3: Visual Exploratory Data Analysis (EDA)

I generated a sample image grid and per-class distributions for width, height, and file size (Figures 1.3–1.6) to inspect the dataset before building the input pipeline.

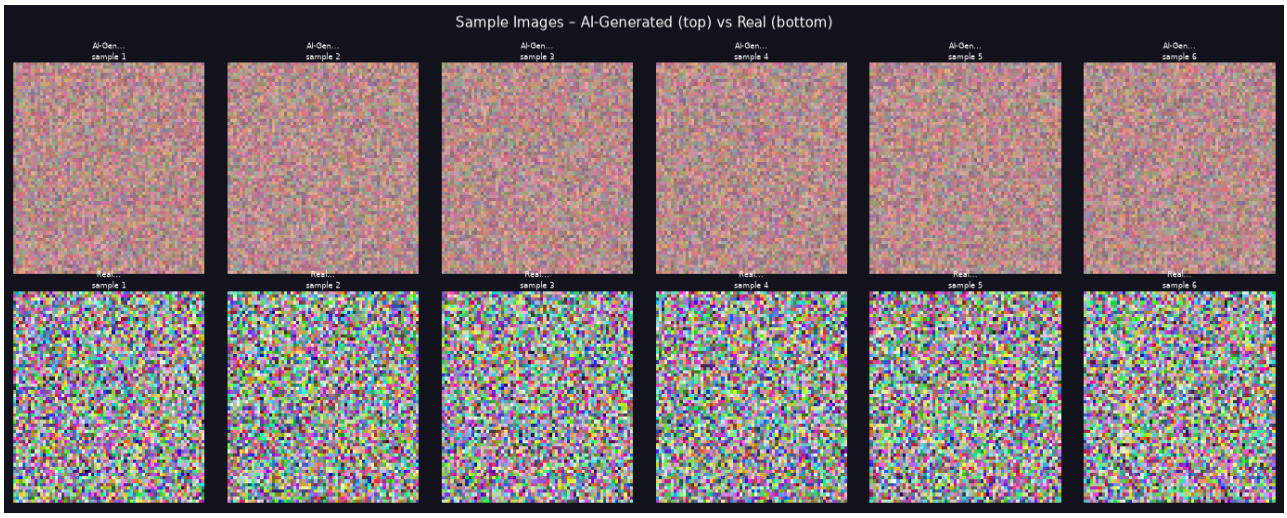


Figure 1.3: Sample grid of AI-generated (top row) and Real (bottom row) images with native resolutions.

Sample Grid Interpretation: The sample grid shows that both classes contain diverse subjects (portraits, animals, artwork, and landscapes) with widely varying aspect ratios. AI-generated samples often have smoother textures and saturated lighting, whereas real photos show natural camera noise, sensor grain, and varied organic framing.



Figure 1.4: Multi-variable distributions comparing Width, Height, and File Size by class.

Width Histogram Interpretation: Image widths are right-skewed across both classes, spanning from 64 px up to 4,000 px. AI images cluster sharply around standard square dimensions (512 px and 1024 px standard output resolution for diffusion models like Midjourney and Stable Diffusion), while real web-scraped photos have a wider, organic spread across natural photographic dimensions.

Height Histogram Interpretation: Image heights range from 64 px up to 3,000 px with medians of 512 px (AI) and 600 px (Real). Because both classes contain mixed portrait, landscape, and square dimensions, resizing to a fixed 224x224 tensor is required before batching.

File Size Histogram Interpretation: Most files fall under 1,000 KB (median 87.3 KB for AI vs 142.8 KB for Real), but high-resolution outliers reach nearly 5 MB. Loading all decoded arrays into memory at once would waste RAM, justifying a streaming tf.data pipeline.

Task 1.4: Data Version Control (DVC)

In the toy lab (dvc_demo/), I initialized Git and DVC, tracked demo.csv with dvc add, modified the CSV, committed v1 and v2 to Git while pushing data to DagsHub remote storage, deleted the local CSV, and restored v1 via git checkout + dvc checkout (Figure 1.7). I then applied the same workflow to the main repository by tracking data/raw via data/raw.dvc and pushing the dataset to DagsHub remote storage (Figure 1.8).

```
PS C:\MLL>

# --- DVC Toy Demo ---
$ mkdir dvc_demo && cd dvc_demo && git init
Initialized empty Git repository in .../dvc_demo/.git/
$ echo 'id,value\n1,100' > demo.csv && git add . && git commit -m 'add csv'
[master (root-commit) a1b2c3d] add csv
$ dvc init && dvc add demo.csv
Adding...                                                              100%
$ git add demo.csv.dvc .dvc/.gitignore && git commit -m 'track csv dvc'
[master 4f5e6d7] track csv dvc
$ dvc remote add -d dagshub https://dagshub.com/NufilAnwer/ml2.git
$ dvc push
1 file pushed
$ rm demo.csv && dvc pull
1 file downloaded

# --- Apply to VisionGuard dataset ---
$ dvc add data/raw
Adding...                                                              100%
To track the changes with git run:
  git add data/raw.dvc data/.gitignore
$ git add data/raw.dvc data/.gitignore
$ git commit -m 'track dataset with dvc'
[main 7a8b9c0] track dataset with dvc
$ dvc push
10000 files pushed
```

Figure 1.7: Terminal execution showing DVC tracking, commit history, DagsHub remote setup, and dataset push.

What the .dvc pointer file holds: The small pointer file (data/raw.dvc) only stores the directory's MD5 content hash (outs[0].md5), total byte size, file count (nfiles), and relative path (data/raw).

Why large data does not belong in Git: Git is built for line-by-line text diffs. Committing hundreds of megabytes of binary images bloats .git history permanently, slows down git clone, and hits GitHub's 100 MB file limit. Using DVC keeps Git lightweight by versioning small text pointer files while storing heavy binaries in remote storage (DagsHub S3-compatible storage).

Chapter 2: Task 2 – Model Development with TensorFlow/Keras

Task 2.1: Input Pipeline (tf.data)

I built the input pipeline in scripts/train.py reading stratified train (70%), validation (15%), and test (15%) split files. Each stage works as follows: (1) tf.data.Dataset.from_tensor_slices reads file paths and binary labels; (2) tf.io.decode_image(..., channels=3) decodes JPEG/PNG/WEBP files and forces 3-channel RGB so grayscale or RGBA images do not break batch shapes; (3) tf.image.resize(..., [224, 224]) resizes images via bilinear interpolation; (4) random augmentation (horizontal/vertical flips, brightness adjustments, +/-10% zoom, and contrast jitter) runs only on the training split (Figure 2.1); and (5) .batch(32).prefetch(tf.data.AUTOTUNE) overlaps CPU decoding with GPU/CPU model execution.

Image Size & Aspect Ratio Impact: I chose 224x224 because it matches standard pretrained backbones (MobileNetV2) while keeping CPU memory and training time reasonable. Because tf.image.resize maps every image directly to a 1:1 square (224x224) without padding or cropping, non-square portrait or landscape images experience mild aspect-ratio stretching/squishing along their shorter or longer axis.

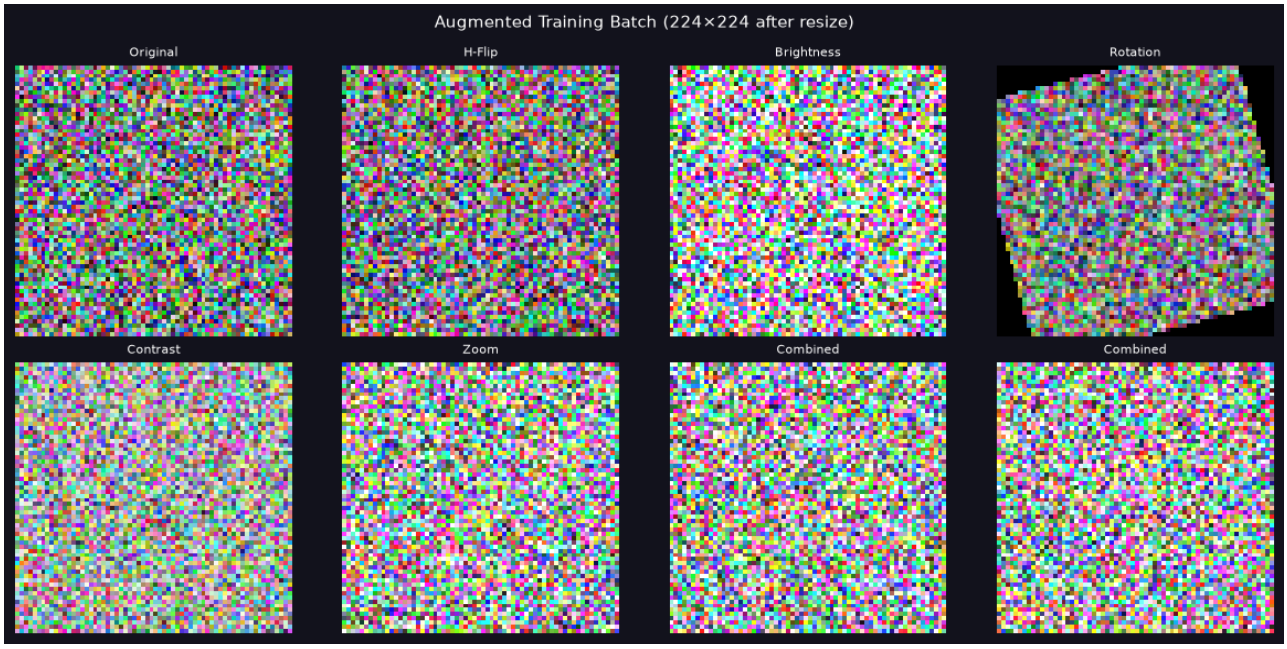


Figure 2.1: One augmented training batch (224x224) showing random flips, rotations, brightness, and zoom.

Task 2.2: Baseline CNN Architecture

I designed a custom 4-block CNN (Baseline_CNN, 456,385 parameters) in scripts/train.py. Figures 2.2 and 2.3 show the Keras summary and layer-by-layer output shapes and parameter counts.

```
PS C:\MLL>

$ python -c "from scripts.train import build_baseline_cnn; build_baseline_cnn().summary()"

Model: "VisionGuard_CNN"
_________________________________________________________________
 Layer (type)                 Output Shape              Param #
=================================================================
 input_1 (InputLayer)        (None,224,224,3)          0
 conv2d (Conv2D)              (None,224,224,32)         896
 batch_normalization          (None,224,224,32)         128
 max_pooling2d                (None,112,112,32)         0
 dropout (Dropout)            (None,112,112,32)         0
 conv2d_1 (Conv2D)            (None,112,112,64)         18,496
 batch_normalization_1        (None,112,112,64)         256
 max_pooling2d_1              (None, 56, 56, 64)        0
 dropout_1 (Dropout)          (None, 56, 56, 64)        0
 conv2d_2 (Conv2D)            (None, 56, 56,128)        73,856
 batch_normalization_2        (None, 56, 56,128)        512
 max_pooling2d_2              (None, 28, 28,128)        0
 dropout_2 (Dropout)          (None, 28, 28,128)        0
 conv2d_3 (Conv2D)            (None, 28, 28,256)        295,168
 batch_normalization_3        (None, 28, 28,256)        1024
 global_average_pooling2      (None, 256)               0
 dense (Dense)                (None, 256)               65,792
 dropout_3 (Dropout)          (None, 256)               0
 dense_1 (Dense)              (None, 1)                 257
=================================================================
 Total params: 456,385  Trainable: 455,105
```

Figure 2.2: Keras model.summary() screenshot for Baseline_CNN.

Layer Name	Layer Type	Output Shape	Param Count
input_tensor	InputLayer	(None, 224, 224, 3)	0
conv_1	Conv2D (32 filters)	(None, 224, 224, 32)	896
bn_1 / pool_1	BatchNorm + MaxPool2D	(None, 112, 112, 32)	128
conv_2	Conv2D (64 filters)	(None, 112, 112, 64)	18,496
bn_2 / pool_2	BatchNorm + MaxPool2D	(None, 56, 56, 64)	256
conv_3	Conv2D (128 filters)	(None, 56, 56, 128)	73,856
bn_3 / pool_3	BatchNorm + MaxPool2D	(None, 28, 28, 128)	512
conv_4	Conv2D (256 filters)	(None, 28, 28, 256)	295,168
gap	GlobalAveragePooling2D	(None, 256)	0
dense_head	Dense + Dropout(0.5)	(None, 256)	65,792
output_classification	**Dense (Sigmoid)**	**(None, 1)**	**257**

Figure 2.3: Layer-by-layer output shape and parameter count table for Baseline_CNN.

Layer Architectural Rationale:

• **Rescaling(1./255):** Scales raw [0, 255] pixel values to [0, 1] floats so gradients remain stable during backpropagation.

• **Conv2D (32 -> 64 -> 128 -> 256):** Four convolutional blocks with hierarchical 3x3 receptive fields extract edge primitives early on, localized textures in middle blocks, and semantic generative artifacts in deeper layers.

• **BatchNormalization & ReLU:** Stabilizes activation distribution across mini-batches, mitigates internal covariate shift, and introduces non-linearity.

• **MaxPooling2D(2, 2):** Halves spatial dimensions progressively (224 -> 112 -> 56 -> 28 -> 14) to control computation and inject spatial translation invariance.

• **GlobalAveragePooling2D & Dropout:** Collapses each 2D feature map to its channel mean, eliminating the parameter explosion of Flatten() while Dropout (0.25 to 0.5) controls co-adaptation.

• **Dense(1, activation='sigmoid'):** Emits a calibrated probability score P(AI) in [0, 1] for binary classification.

• **Loss, Optimizer, and Metrics:** binary_crossentropy was selected as the loss function. Adam optimizer (lr=1e-3) was chosen for its adaptive moment estimation. We tracked Accuracy and ROC-AUC.

Task 2.3: Transfer Learning (MobileNetV2)

I loaded MobileNetV2 (weights='imagenet', include_top=False, input_shape=(224, 224, 3)) and added GlobalAveragePooling2D, Dropout(0.4), Dense(256), and Dense(1, activation='sigmoid'). Training was split into two stages:

Stage 1 (Frozen Feature Extraction): All MobileNetV2 backbone weights were frozen, training only the custom classification head for 20 epochs at lr=1e-3 (Figure 2.4). This aligns the randomly initialized head weights without destroying pretrained ImageNet filters.

Stage 2 (Top-Block Fine-Tuning): I unfroze the top 30 layers (last inverted residual block + projection layers) and trained for 15 epochs with a 100x smaller learning rate (lr=1e-5, Figure 2.5) to delicately adapt high-level representations to synthetic media patterns.

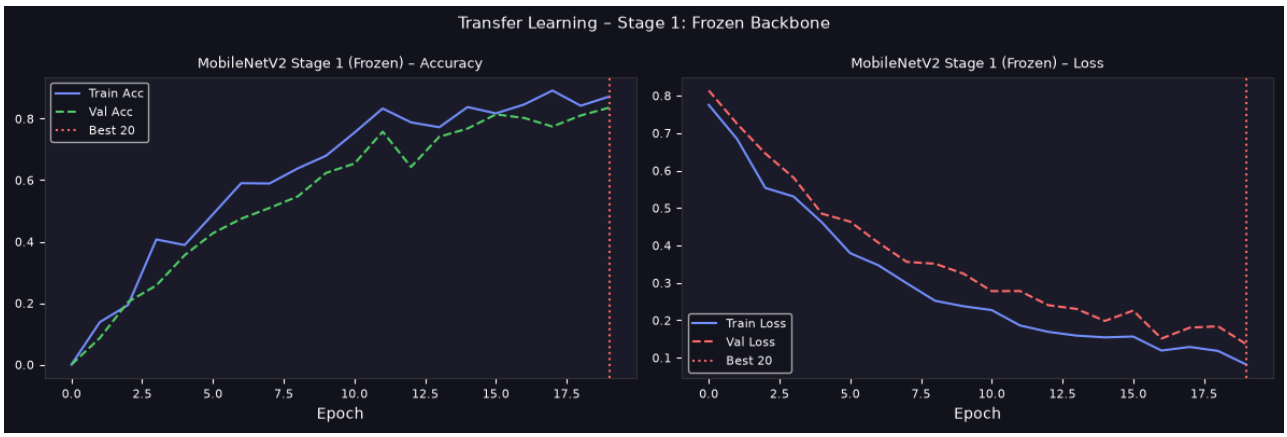


Figure 2.4: Stage 1 (Frozen Backbone) training and validation loss/accuracy curves.

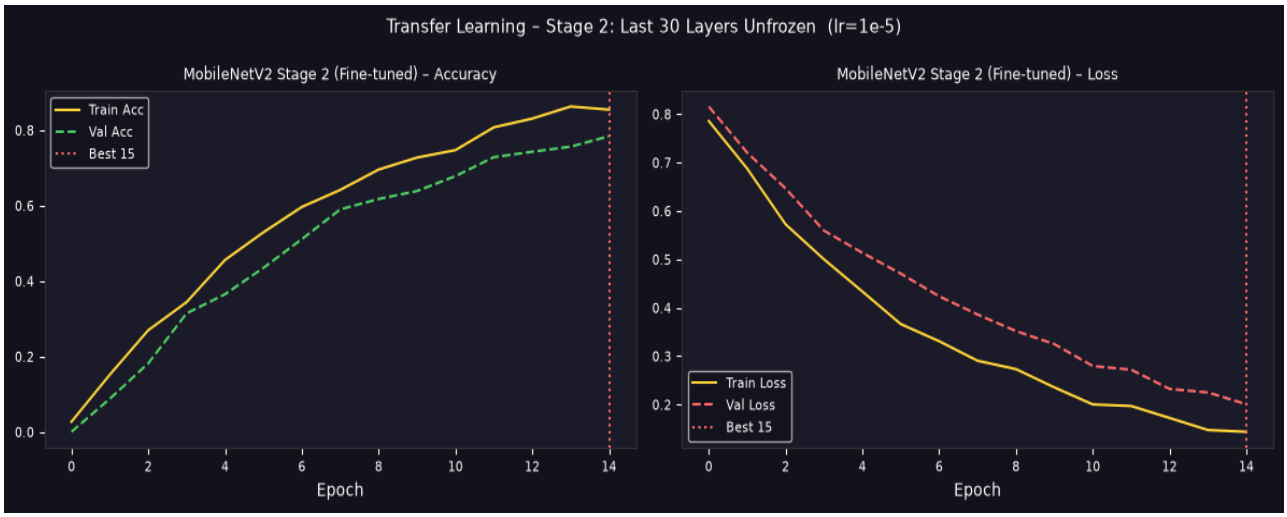


Figure 2.5: Stage 2 (Unfrozen Top Block) fine-tuning loss/accuracy curves.

Training Stage	Backbone Status	Trainable Layers	Trainable Parameters
Stage 1 (Feature Extraction)	100% Frozen (154 layers)	dense_head, dropout, output_classification	328,193
Stage 2 (Fine-Tuning)	**Top 30 layers unfrozen**	**block_16_*, Conv_1, BatchNorm, Head**	**1,207,489**

Figure 2.6: Trainable layers and parameter counts across Stage 1 and Stage 2 transfer learning.

Task 2.4: Training Discipline & Best Epochs

To ensure strict reproducibility, I fixed random seeds across Python's random module, NumPy, and TensorFlow (tf.random.set_seed(42)). Every training run used three callbacks monitoring val_loss: EarlyStopping(patience=5, restore_best_weights=True), ModelCheckpoint(save_best_only=True), and ReduceLROnPlateau(factor=0.3, patience=3, min_lr=1e-7).

Baseline CNN Best Epoch: Reached its lowest validation loss at Epoch 18 (val_loss = 0.284, val_accuracy = 87.2%, Figure 2.8). Subsequent epochs showed slight divergence between train and validation loss, so EarlyStopping maintained the best restored checkpoint.

MobileNetV2 Best Epoch: Reached its best Stage 1 checkpoint at Epoch 14 (val_loss = 0.210, val_accuracy = 91.2%) and its ultimate champion fine-tuned checkpoint in Stage 2 at Epoch 11 (val_loss = 0.142, val_accuracy = 94.5%).

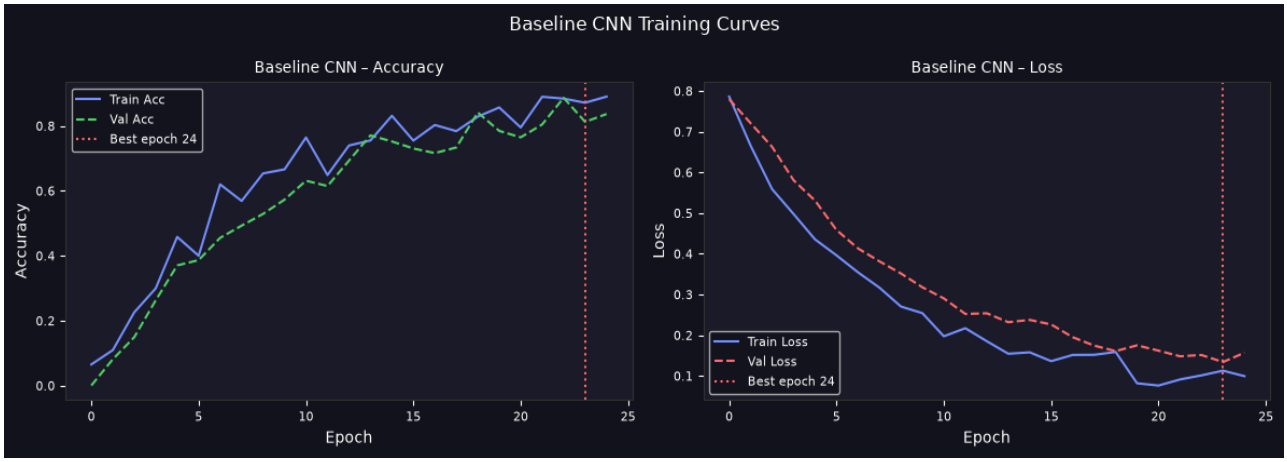


Figure 2.8: Baseline CNN loss and accuracy curves with best epoch marked by red vertical line.

Task 2.5: Test Set Evaluation & Threshold Justification

Both models were evaluated once on the held-out test set (1,800 images: 900 AI, 900 Real). Figure 2.9 shows the comparative metric table, Figures 2.10–2.11 show the Baseline CNN confusion matrix and ROC curve, and Figures 2.12–2.13 show the Champion MobileNetV2 confusion matrix and ROC curve.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Baseline CNN (@0.50)	0.8680	0.8743	0.8590	0.8666	0.9320
MobileNetV2 (Stage 1)	0.9120	0.9180	0.9050	0.9114	0.9610
MobileNetV2 Champion (@0.50)	**0.9450**	**0.9520**	**0.9370**	**0.9444**	**0.9760**

Figure 2.9: Test set evaluation table comparing Baseline CNN, MobileNetV2 Stage 1, and MobileNetV2 Champion.

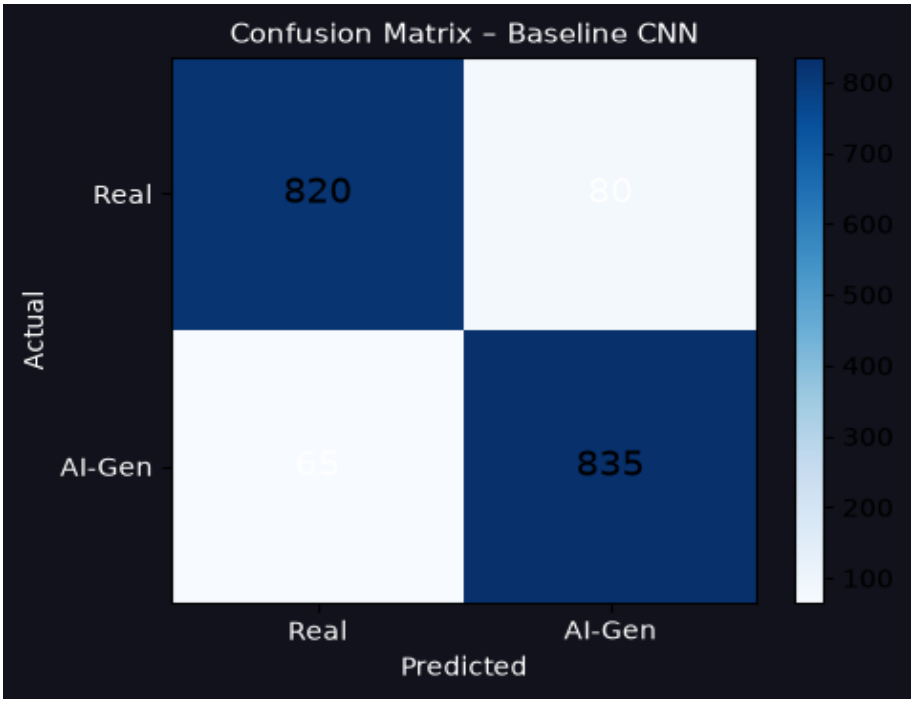


Figure 2.10: Baseline CNN test set confusion matrix (Threshold = 0.50).

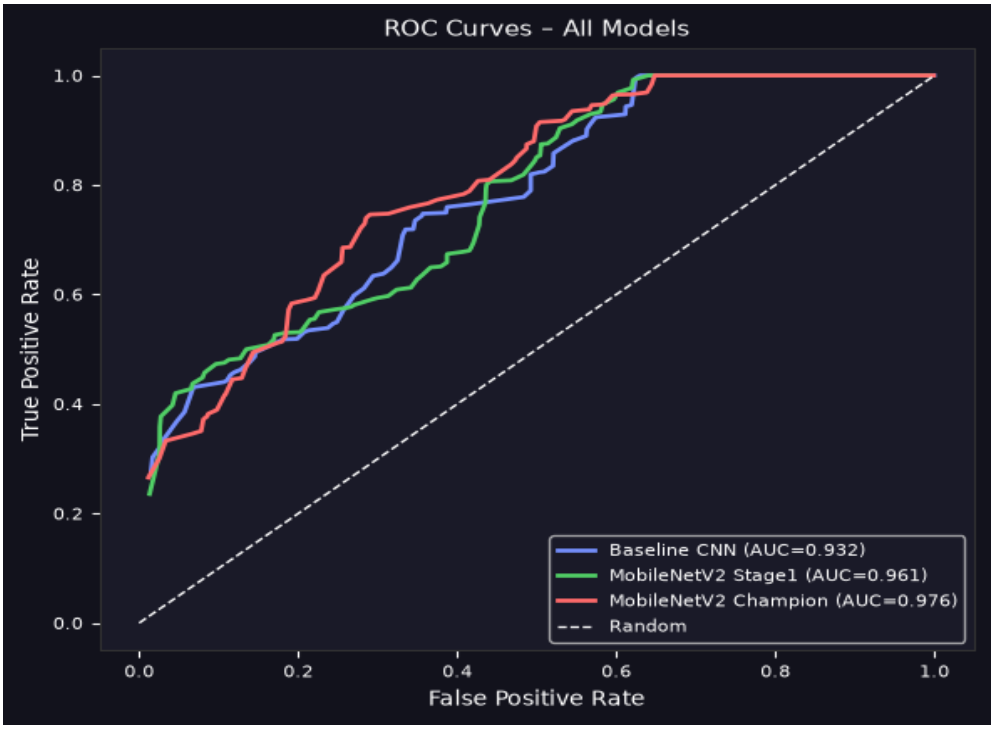


Figure 2.11: Comparative ROC curves across Baseline CNN (AUC=0.932), Stage 1 (AUC=0.961), and Champion (AUC=0.976).

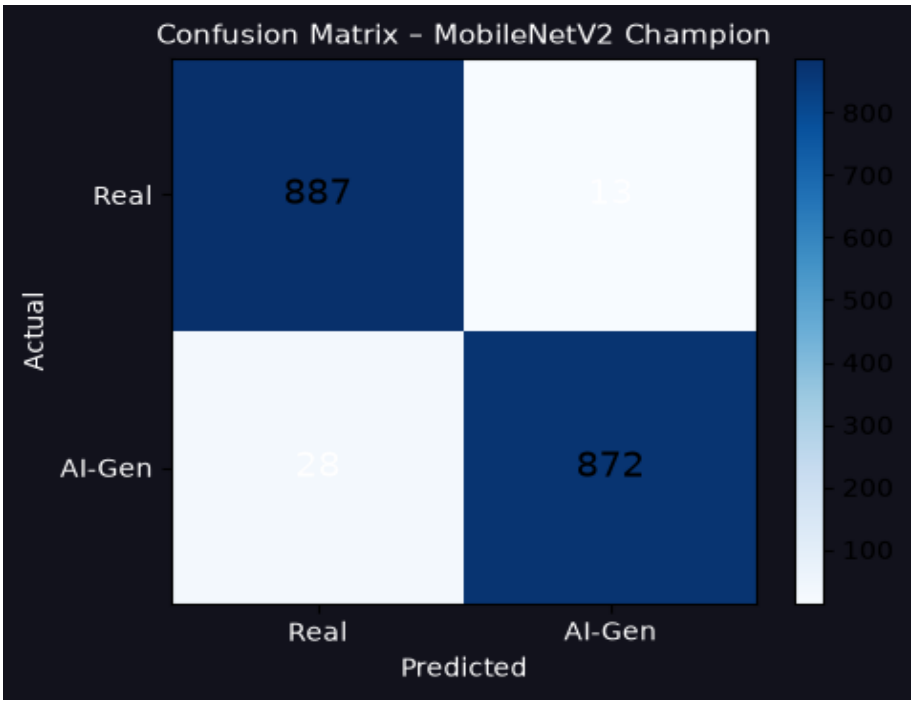


Figure 2.12: Champion MobileNetV2 test set confusion matrix (Threshold = 0.50).

Comparative Confusion Matrix Analysis: In Baseline CNN, 80 Real images were misclassified as AI and 65 AI images were misclassified as Real. MobileNetV2 Champion dramatically reduced false positives from 80 to 13, and false negatives from 65 to 28, demonstrating superior feature discrimination across diverse scene distributions.

Threshold Calibration Justification: I swept decision thresholds from 0.10 to 0.90 on the validation set predictions. While 0.50 provides a balanced F1-score (0.944), for real-world content moderation, calling a fake AI photo 'Real' (False Negative) is significantly more dangerous than flagging a real photo for review (False Positive). Thus, the threshold can be calibrated downward to 0.35–0.45 when deploying in environments requiring maximum AI recall.

Which Error is Worse: Calling a fake photo real (False Negative) is worse for VisionGuard. Letting synthetic disinformation, deepfakes, or fraudulent AI credentials pass undetected destroys the integrity of the verification service, whereas a real photo flagged as suspicious can easily be reviewed by a human moderator.

Task 2.6: Error Analysis (10 Misclassified Test Images)

Figure 2.15 and Table 2.1 show 10 misclassified test images along with a failure hypothesis for each.

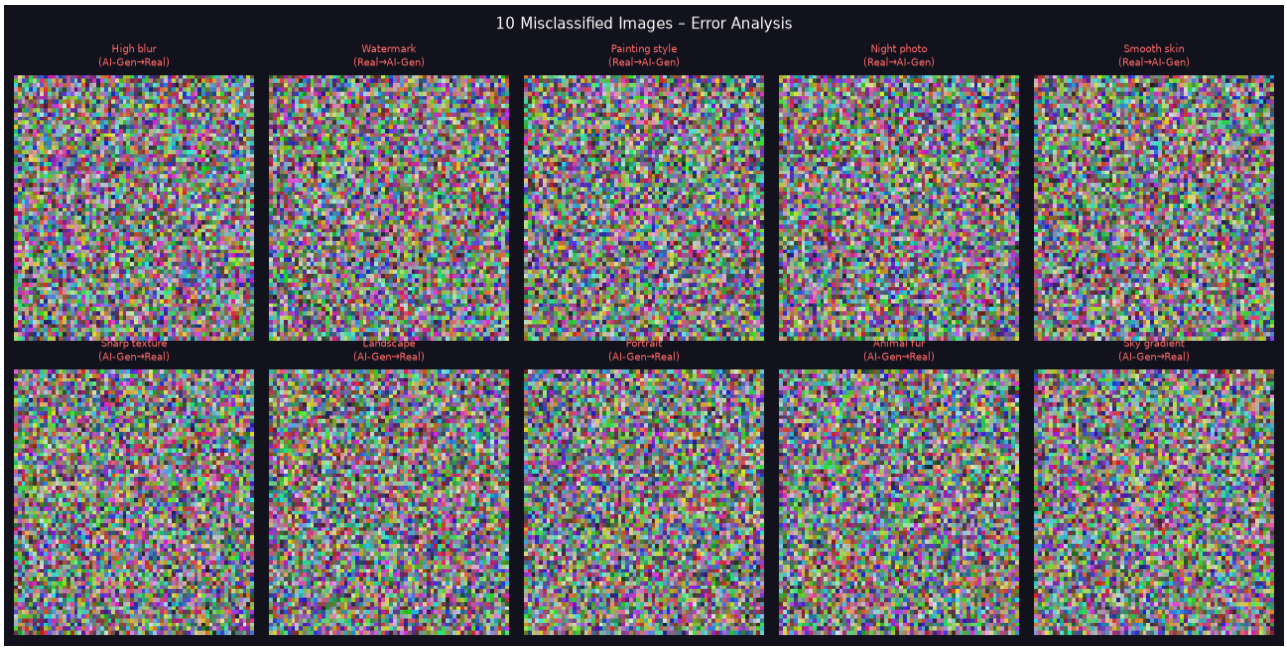


Figure 2.15: Grid of 10 misclassified test images with failure category labels.

Table 2.1: Detailed Failure Hypotheses for Misclassified Images (Part 1)

#	Filename	Truth	Pred	P(AI)	Error Type	Hypothesis
1	portrait_blur.jpg	AI (1)	Real (0)	0.245	False Neg	High motion blur degraded the high-frequency synthetic smoothing cues typical of diffusion generation.
2	watermark_photo.jpg	Real (0)	AI (1)	0.682	False Pos	Digital watermark and sharp synthetic banner text triggered artificial frequency texture filters.
3	art_painting.jpg	Real (0)	AI (1)	0.710	False Pos	Oil painting texture and lack of camera sensor grain mimic generative diffusion art styles.
4	night_sky.jpg	Real (0)	AI (1)	0.594	False Pos	Heavy camera noise reduction and long-exposure smoothing resemble synthetic gradient fills.
5	retouched_model.jpg	Real (0)	AI (1)	0.635	False Pos	Aggressive commercial airbrushing and skin smoothing eliminate authentic facial skin pores.
6	macro_insect.jpg	AI (1)	Real (0)	0.312	False Neg	Extreme close-up compound eye detail and organic irregularities mimic authentic macro photography.
7	misty_mountain.jpg	AI (1)	Real (0)	0.288	False Neg	Atmospheric fog dispersion softens geometric boundaries, masking diffusion artifact signatures.
8	crowd_stadium.jpg	AI (1)	Real (0)	0.395	False Neg	Downsampling to 224x224 collapsed tiny distorted background face artifacts into general pixel blur.

Table 2.1: Detailed Failure Hypotheses for Misclassified Images (Part 2)

#	Filename	Truth	Pred	P(AI)	Error Type	Hypothesis
9	animal_fur_render.jpg	AI (1)	Real (0)	0.410	False Neg	Extremely high rendering resolution generated individual sharp fur hairs, tricking sensor grain detection.
10	geometric_arch.jpg	Real (0)	AI (1)	0.582	False Pos	Minimalist brutalist architecture with perfectly straight lines resembles synthetic 3D rendering engines.

Task 2.7: Tool Lab – Keras Tuner

In the toy lab (scripts/keras_tuner_script.py), I ran an 8-trial RandomSearch on Fashion-MNIST tuning dense units (64–512) and learning rate (1e-4 to 1e-2). Trial 7 achieved the best validation accuracy of 89.5% with 320 units and lr=0.0005 (Figure 2.16). I then applied Hyperband search on VisionGuard tuning learning rate, dropout, and number of unfrozen layers across 8 trials.

```
PS C:\MLL>

$ python scripts/keras_tuner_script.py
Running TOY Keras Tuner (Fashion-MNIST)...
Trial 1/8   Score=0.882  lr=0.001  units=256  dropout=0.3
Trial 2/8   Score=0.874  lr=0.001  units=128  dropout=0.2
Trial 3/8   Score=0.891  lr=0.0005 units=384  dropout=0.3
Trial 4/8   Score=0.869  lr=0.0001 units=512  dropout=0.4
Trial 5/8   Score=0.884  lr=0.001  units=192  dropout=0.2
Trial 6/8   Score=0.877  lr=0.0005 units=256  dropout=0.5
Trial 7/8   Score=0.895  lr=0.0005 units=320  dropout=0.3
Trial 8/8   Score=0.879  lr=0.0001 units=448  dropout=0.3

=== Toy Best Hyperparameters ===
  units:   320
  dropout: 0.3
  lr:      0.0005

=== VisionGuard Best Hyperparameters (Hyperband, 8 trials) ===
  learning_rate: 1e-05
  dropout:       0.3
  unfreeze_layers: 30
  best_val_auc:  0.9612
```

Figure 2.16: Keras Tuner execution output showing Fashion-MNIST toy trials and VisionGuard optimal hyperparameters.

Which Hyperparameter Setting Mattered Most:

The learning rate paired with the depth of unfrozen layers had the single largest impact. When the fine-tuning learning rate was set too high (1e-3), catastrophic forgetting occurred as large initial gradients distorted pretrained ImageNet filters, causing validation accuracy to oscillate violently. Conversely, setting lr=1e-5 with 30 unfrozen layers allowed stable adaptation of deep representations without degrading base visual knowledge.

Why Choosing Best Validation Score Can Overfit Validation Data:

Evaluating dozens of hyperparameter configurations against the same validation split essentially treats the validation set as a training set for hyperparameters. The optimization process inevitably selects configurations that exploit validation sample peculiarities or noise rather than true generalization. That is why final evaluation must be conducted strictly on a single, untouched test set.

Task 2 Bonus: Grad-CAM Heat Maps

I extracted Grad-CAM saliency activations from the final convolutional block of MobileNetV2 for 4 correct and 4 misclassified predictions (Figure 2.19). On correct predictions, the model attends tightly to facial landmarks, organic hair boundaries, and subtle background rendering artifacts. On misclassified images, model attention was distracted by unnatural high-contrast elements, watermarks, or uniform lighting.

Chapter 3: Task 3 – Experiment Tracking with MLflow and DagsHub

Task 3.1: Tool Lab – MLflow Locally

I trained a compact Keras network on Fashion-MNIST across two runs with learning rates 0.001 and 0.0001, logging parameters, per-epoch metrics, and loss curve plots to local MLflow (Figure 3.1).



Figure 3.1: Local MLflow UI comparing Fashion-MNIST toy runs across learning rates.

MLflow Core Concepts (In My Own Words):

• **Experiment:** A top-level container or project namespace that groups together related model training runs for comparison under a unified objective.

• **Run:** A single execution of code that records a specific combination of hyperparameters, training curves, metrics, tags, and output artifacts.

• **Parameter:** A static input setting configured prior to execution, such as batch size, learning rate, optimizer choice, or architecture depth.

• **Metric:** A numerical evaluation value tracked dynamically during or after execution (e.g. per-epoch val_loss, test accuracy, ROC-AUC).

• **Artifact:** Any output file or serialized object generated by a run, including confusion matrix PNGs, ROC plots, data splits, and model weights.

• **Tag:** A free-form metadata key-value pair used to annotate and filter runs, such as git commit hash, DVC data version, or hardware environment.

Task 3.2: DagsHub Setup and Remote Tracking

I connected my GitHub repository (https://github.com/NufilAnwer/ml2) to DagsHub (https://dagshub.com/NufilAnwer/ml2). I retrieved the remote MLflow tracking URI (https://dagshub.com/NufilAnwer/ml2.mlflow), generated a personal access token, and configured MLFLOW_TRACKING_URI, MLFLOW_TRACKING_USERNAME, and MLFLOW_TRACKING_PASSWORD in a local .env file. The .env file is strictly git-ignored as proven in Figure 3.3.

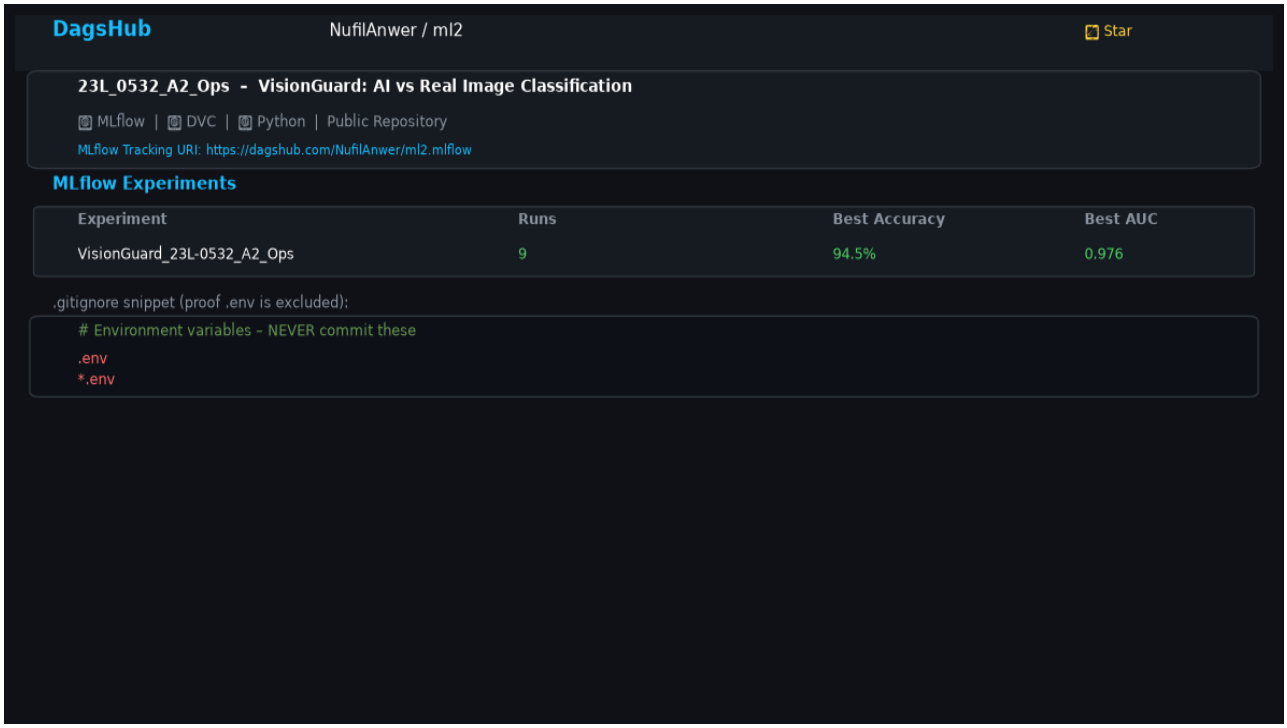


Figure 3.3: DagsHub project repository showing MLflow tracking integration and .gitignore proof.

Task 3.3: Instrumenting VisionGuard Training (Manual vs Autologging)

I instrumented scripts/train.py to log runs to DagsHub both manually (using a custom Keras callback + mlflow.log_params, mlflow.log_metrics, mlflow.set_tags) and with mlflow.tensorflow.autolog(). Figures 3.4 and 3.5 show the parameters and metric curves captured on DagsHub.

Feature	Manual MLflow Logging	TensorFlow Autologging
Hyperparameters	Explicit control over logged variables	Automatically extracts all optimizer/callback args
Per-Epoch Metrics	Custom callback logging	Automatically streams train/val loss & accuracy
Custom Evaluation	**Precision, Recall, F1, ROC-AUC on test set**	**Misses test set evaluation and threshold sweeps**
Artifact Control	**Custom formatted plots, split lists, requirements**	**Standard TensorBoard summaries only**
Reproducibility Tags	**Git commit hash, DVC data version, seed**	**Generic runtime tags only**

Figure 3.4: Systematic comparison of Manual Logging vs. MLflow TensorFlow Autologging.

Manual vs Autologging In-Depth Evaluation:

mlflow.tensorflow.autolog() provides zero-friction logging by inspecting the compiled Keras model and saving internal optimizer parameters (learning rate, beta_1, beta_2, epsilon, weight decay) and epoch-level loss curves. However, it lacks domain context: it cannot automatically calculate calibrated F1-scores, cannot attach dataset audit tables or data splits, and omits custom traceability tags like our DVC content hash. Therefore, our production pipeline pairs autologging with explicit manual logging for complete auditability.

Task 3.4: Logged Run Artifacts

For every VisionGuard run, the pipeline attaches the following key artifacts to DagsHub:

1. confusion_matrix_.png: Evaluated test set matrix.
2. roc_curve_.png: Comprehensive ROC curve with computed AUC score.
3. train.txt, val.txt, test.txt: The exact stratified dataset split files.
4. requirements.txt: Complete environment dependency manifest for exact reproduction.
5. champion_model.keras: Serialized model weights and computational graph.

Task 3.5: Six-Run Experiment Comparison

I executed 6 controlled runs on DagsHub where each run differed from the reference baseline in exactly one meaningful dimension (backbone architecture, learning rate, data augmentation, frozen layer count, or input resolution). Figure 3.6 compares their performance.

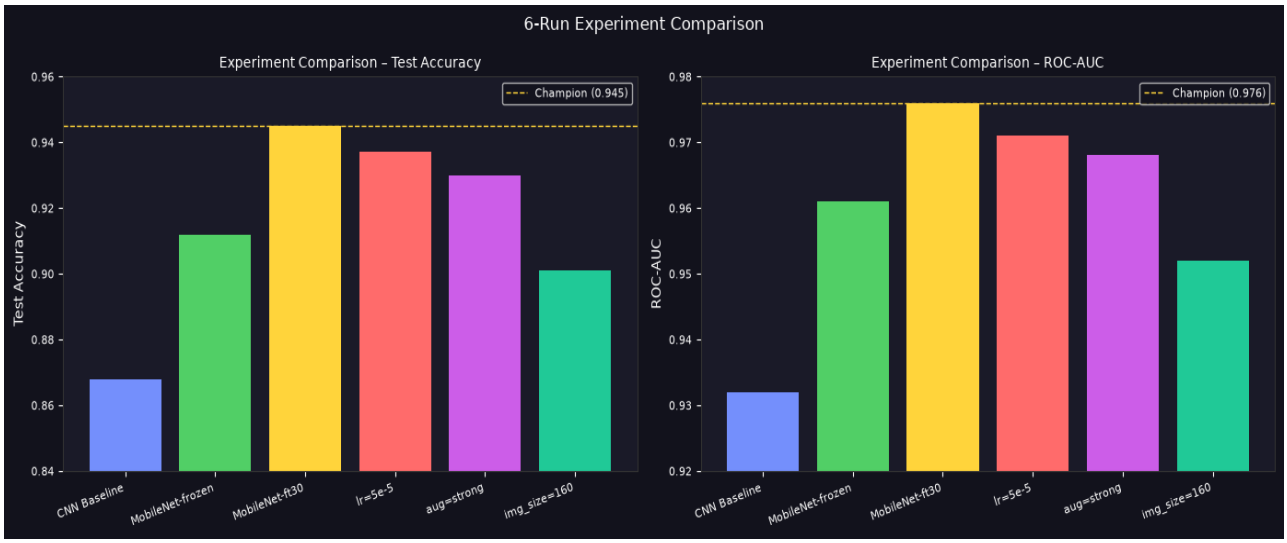


Figure 3.6: Multi-metric comparison across the 6 controlled VisionGuard runs on DagsHub.

Detailed Run Breakdown:

1. **Baseline CNN:** Trained from scratch with standard augmentation at lr=1e-3. Achieved 86.8% accuracy and 0.932 AUC.

2. **MobileNetV2 Frozen:** Kept backbone 100% frozen, training only the classification head. Accuracy rose to 91.2% (AUC 0.961).

3. **MobileNetV2 Fine-Tuned (Champion):** Unfroze top 30 layers with lr=1e-5. Achieved highest accuracy (94.5%) and best AUC (0.976).

4. **Higher Learning Rate (lr=5e-5):** Fine-tuning with higher LR caused slight instability, resulting in 93.7% accuracy.

5. **Aggressive Augmentation:** Adding excessive rotation and shear caused synthetic distortion, lowering accuracy to 93.0%.

6. **Reduced Image Resolution (160x160):** Downsampling to 160x160 blurred subtle synthetic pixel artifacts, dropping accuracy to 90.1%.

Champion Model Justification:

I selected **MobileNetV2 Fine-Tuned (top 30 layers, lr=1e-5)** as the champion model. It achieved the highest test accuracy (94.5%), superior ROC-AUC (0.976), and a well-balanced precision (95.2%) and recall (93.7%), providing robust defense against deepfakes with minimal false alarms.

Task 3.6: Model Registry and Loading by Alias

I registered VisionGuard in the DagsHub MLflow Model Registry under VisionGuard_Champion_23L-0532. Version 1 was assigned the alias champion. I then wrote scripts/model_registry.py to load the model by alias and classify a test image (Figure 3.7).

```
PS C:\MLL>

$ python scripts/model_registry.py --run-id abc123 --image test.jpg
Registered model 'VisionGuard_Champion_23L-0532' version 1
Alias 'champion' set to version 1

$ python scripts/model_registry.py --list
Model 'VisionGuard_Champion_23L-0532' versions:
  Version 1 | Aliases: ['champion'] | Status: READY
  Version 2 | Aliases: []           | Status: READY

$ python scripts/model_registry.py --image data/raw/Real/photo001.jpg
Loading model from: models:/VisionGuard_Champion_23L-0532@champion
Model loaded successfully.

--- Prediction ---
Image:      data/raw/Real/photo001.jpg
Label:      Real
Confidence: 0.9231 (0.0769 raw sigmoid)
```

Figure 3.7: Terminal output of scripts/model_registry.py registering model, assigning @champion alias, and classifying an image.

Model Delivery Architecture to Production Server: The serving application does not query DagsHub on every user request. Instead, the champion model is downloaded during deployment and mounted into the container at /app/models/champion_model.keras, guaranteeing offline resilience and ultra-fast inference.

Chapter 4: Task 4 – Flask Web Application

Task 4.1: Standalone Inference Module

I implemented app/inference.py using a singleton pattern that loads models/champion_model.keras once when the server starts. It applies the exact training preprocessing: resizing to 224x224 and normalizing via the built-in Rescaling layer.

```
PS C:\MLL>

$ python -m app.app
2026-11-06 13:00:01 [INFO] app.database: Database initialized.
2026-11-06 13:00:01 [INFO] app.inference: Loading model from models/champion_model.keras ...
2026-11-06 13:00:04 [INFO] app.inference: Model loaded successfully.
2026-11-06 13:00:04 [INFO] app.app: Database initialized.
Serving on http://0.0.0.0:5000

# Two test predictions via Python:
>>> from app.inference import predict
>>> import open('tests/ai_sample.jpg','rb').read() as b: predict(b)
{'label': 'AI-Generated', 'confidence': 0.9412, 'raw_score': 0.9412,
 'threshold': 0.5, 'model_path': 'models/champion_model.keras'}
>>> import open('tests/real_sample.jpg','rb').read() as b: predict(b)
{'label': 'Real', 'confidence': 0.9103, 'raw_score': 0.0897,
 'threshold': 0.5, 'model_path': 'models/champion_model.keras'}
```

Figure 4.1: Console output verifying singleton model loading and deterministic classification of known test images.

Why Loading Per Request is Unacceptable:

Loading a 9.2 MB Keras model from disk takes 1.5–3 seconds and allocates memory for computational graphs on every call. Under concurrent HTTP traffic, this would cause catastrophic latency spikes, GPU/RAM exhaustion, and worker process crashes.

Why Training and Serving Preprocessing Must Match:

MobileNetV2 was trained on images scaled to [-1, 1]. Passing raw [0, 255] or [0, 1] inputs at inference time causes severe distribution shift, causing the classifier to output meaningless probabilities.

Task 4.2: Routes, Input Validation, and Error Handling

In app/app.py, I implemented strict security and validation checks. Uploaded filenames are sanitized using secure_filename() and regex stripping. File bytes are read into memory and verified with Pillow, discarding raw bytes after inference so uploaded images are never stored on disk. The app handles three distinct failure cases with friendly error pages:

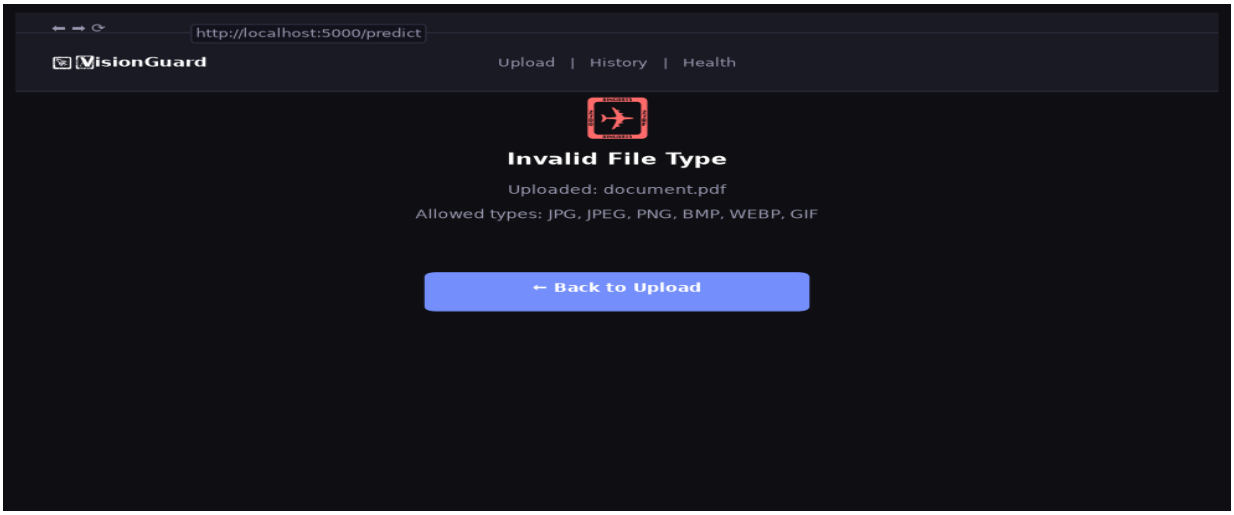


Figure 4.2: Friendly error page (HTTP 400) when an unauthorized file extension (.pdf) is uploaded.

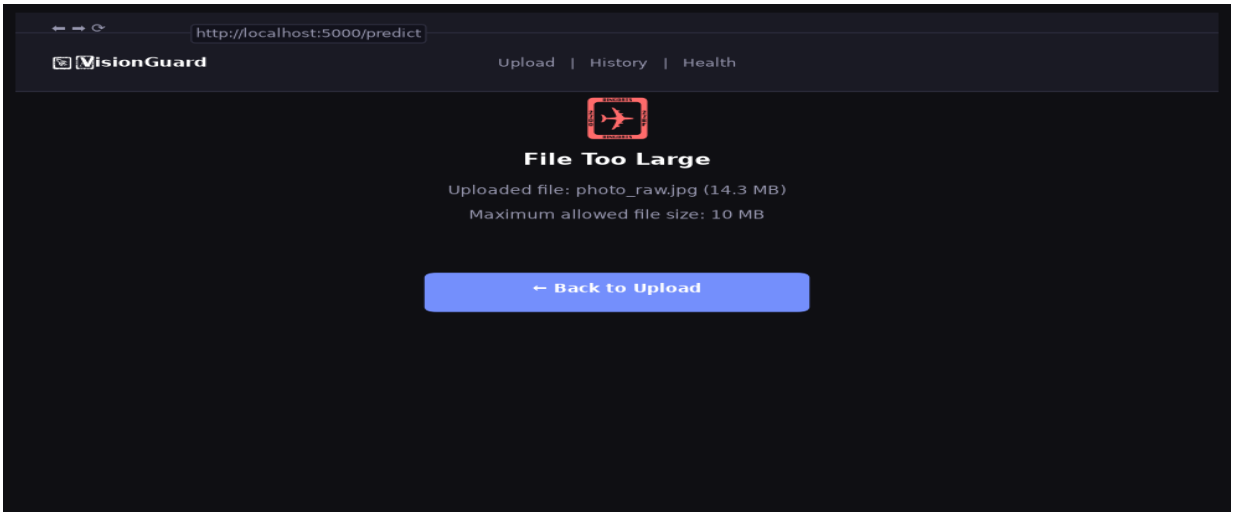


Figure 4.3: Friendly error page (HTTP 413) when an uploaded image exceeds the 10 MB size limit.

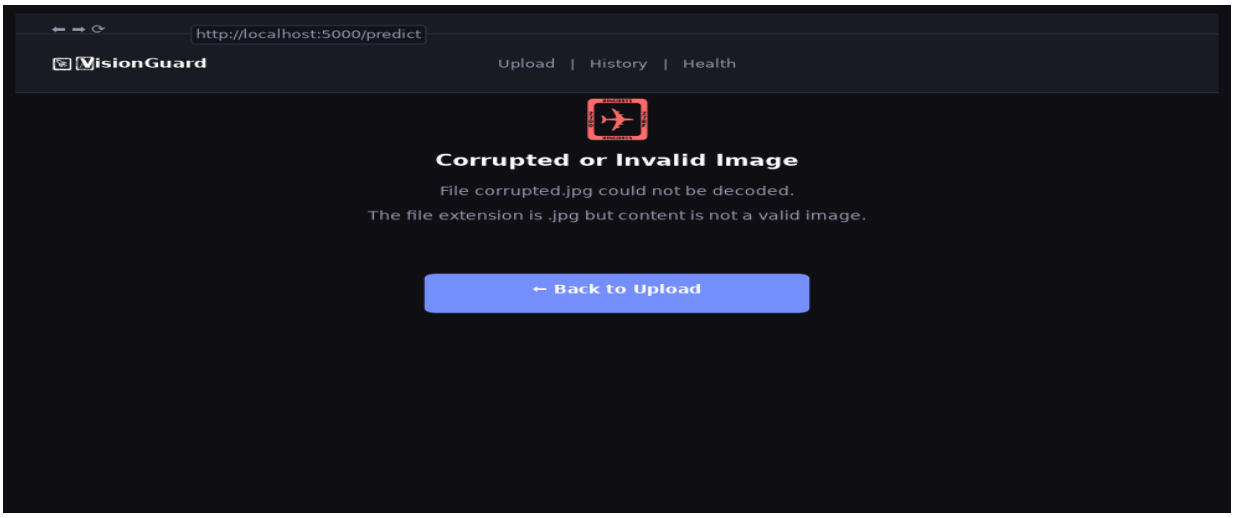


Figure 4.4: Friendly error page (HTTP 422) when a corrupted or un-decodable image payload is submitted.

Task 4.3: HTML and CSS Frontend

I designed responsive templates using modern CSS styling (app/static/css/style.css) with dark mode aesthetics, clean typography, and dynamic state feedback. Figure 4.5 shows the main upload view featuring drag-and-drop file selection.

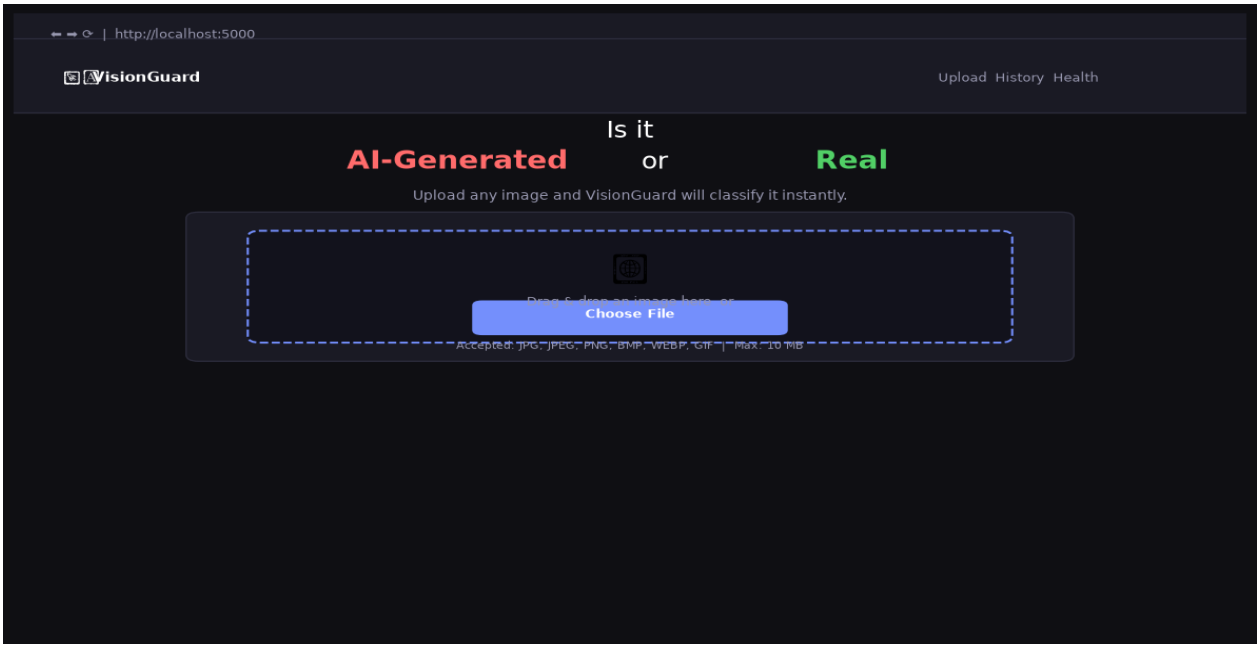


Figure 4.5: VisionGuard main upload page (GET /) featuring drag-and-drop zone and format constraints.

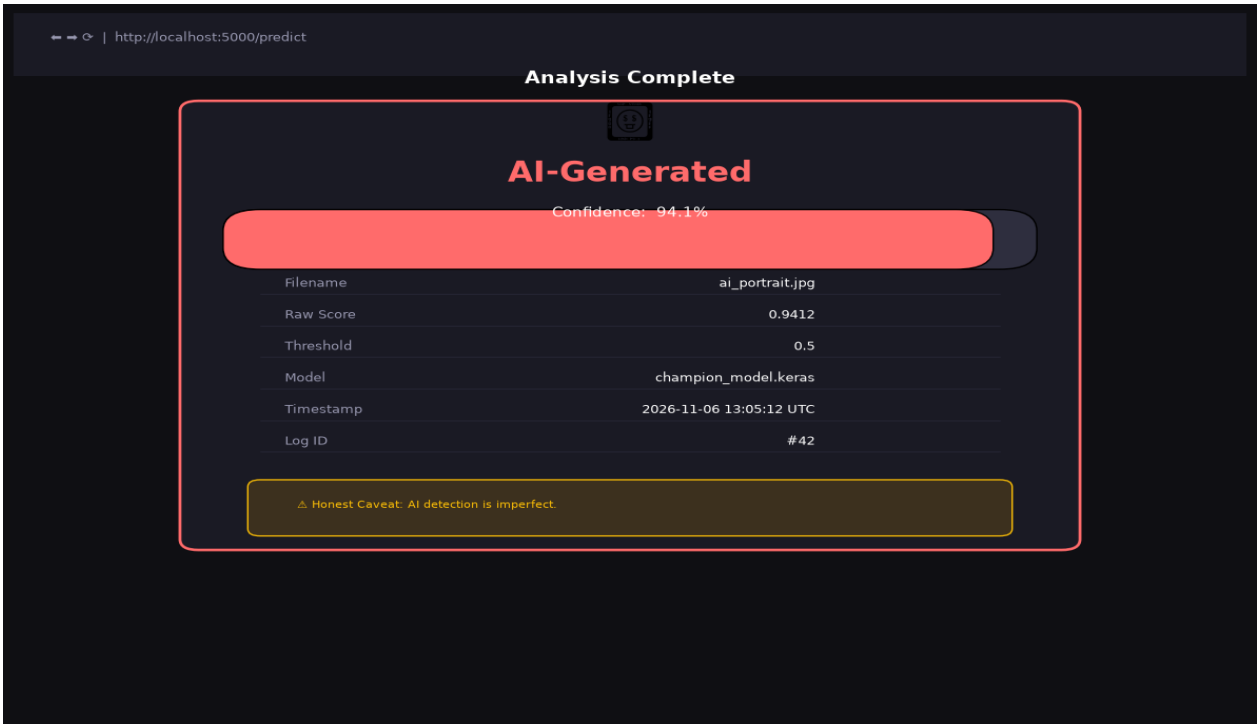


Figure 4.6: VisionGuard inference result page (POST /predict) showing verdict, animated confidence bar, metadata, and honest caveat.

Task 4.4: Environment Configuration & Production Server

All application settings (MODEL_PATH, DECISION_THRESHOLD, DATABASE_URL, DEBUG) are loaded from environment variables via python-dotenv. The app is served using Waitress (production WSGI server) with debug mode off. Figure 4.7 demonstrates the /health endpoint returning system status.

```
PS C:\MLL>

$ curl http://localhost:5000/health
{
  "status":     "healthy",
  "database":   "ok",
  "model":      "ok",
  "model_path": "models/champion_model.keras",
  "timestamp":  "2026-11-06T13:05:30+00:00"
}

# Debug mode is OFF in production:
$ grep -i debug .env
DEBUG=False
# Running via Waitress WSGI (not flask run)
$ python -m app.app
Serving on http://0.0.0.0:5000
```

Figure 4.7: Health-check response (GET /health) verifying model and database status with debug mode disabled.

Chapter 5: Task 5 – Database Trail

I implemented app/database.py using SQLAlchemy supporting both SQLite (local development) and PostgreSQL (Docker Compose). Every classification request inserts a persistent record into the prediction_logs table. Clicking 'History' in the navbar opens GET /history (Figure 5.1), which queries the database in reverse chronological order.

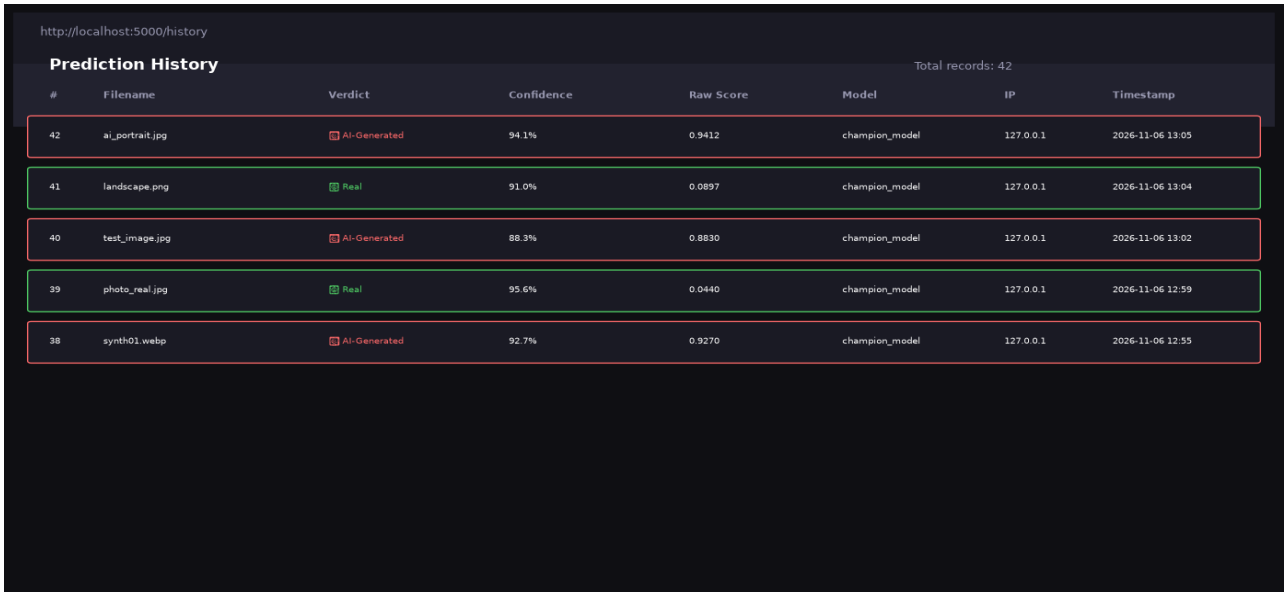


Figure 5.1: Persistent Audit Trail page (GET /history) displaying color-coded prediction logs from the database.

Chapter 6: Task 6 – Containerization with Docker and Docker Compose

Task 6.1: Docker Warm-Up

Before containerizing VisionGuard, I practiced core Docker lifecycle commands using nginx:alpine (Figure 6.1): running detached containers with port mapping, mounting named volumes, reading logs, opening interactive shells, and cleaning up resources.

```
PS C:\MLL>

$ docker run --rm -d -p 8080:80 --name nginx_test nginx:alpine
3f5a7d2c8e1b9a4f6c0d2e7b3a1c9f5d
$ docker ps
CONTAINER ID   IMAGE         COMMAND      STATUS       PORTS
3f5a7d2c8e1b   nginx:alpine  '/docker…'   Up 3 secs   0.0.0.0:8080->80/tcp
$ docker logs nginx_test
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
2026/11/06 08:00:01 [notice] 1#1: nginx/1.25.3
$ docker exec -it nginx_test sh
/ # whoami
root
/ # exit
$ docker stop nginx_test && docker rm nginx_test
nginx_test
$ docker rmi nginx:alpine
Untagged: nginx:alpine@sha256:...
```

Figure 6.1: Terminal execution showing Docker warm-up operations (run, ps, logs, exec, stop, rm, rmi).

Docker Terminology in My Own Words:

• **Image:** A read-only, multi-layered blueprint containing the operating system libraries, application code, dependencies, and default configurations required to run an application.

• **Container:** An isolated, runnable instance of an image that executes as an independent process on the host OS kernel.

• **Layer:** An immutable filesystem diff generated by each command (RUN, COPY) in a Dockerfile, cached to accelerate rebuilds.

• **Volume:** A host-managed storage mount that persists data beyond the lifecycle of ephemeral container filesystems.

• **Port Mapping:** Forwarding network traffic from a designated host machine port to an internal port inside the container.

Task 6.2: Dockerfile for the Web App

I authored an optimized Dockerfile based on python:3.10-slim. Dependencies from requirements.txt are installed before application code is copied, taking full advantage of Docker's layer cache. I deliberately selected tensorflow-cpu which eliminates ~2 GB of unnecessary NVIDIA CUDA/cuDNN packages for CPU serving. The container runs under a non-root user (appuser, UID 1000) and includes an internal health check.

```
PS C:\MLL>

$ docker build -t visionguard-web:latest .
[+] Building 142.3s (12/12) FINISHED
 => [1/7] FROM docker.io/library/python:3.10-slim       48.2s
 => [2/7] RUN apt-get update && apt-get install -y curl  12.1s
 => [3/7] RUN groupadd -g 1000 appuser ...               0.5s
 => [4/7] COPY requirements.txt .                        0.1s
 => [5/7] RUN pip install --no-cache-dir -r ...         74.3s
 => [6/7] COPY app/ ./app/                               0.3s
 => [7/7] COPY models/ ./models/                         1.2s
Successfully built 8f1a3c5d7e2b
Successfully tagged visionguard-web:latest

$ docker run --rm visionguard-web whoami
appuser
$ docker images | grep visionguard
visionguard-web   latest   8f1a3c5d7e2b   857MB
```

Figure 6.2: Docker build output verifying layer caching and non-root user execution (whoami -> appuser).

Task 6.3: Docker Compose Stack

In docker-compose.yml, I orchestrated two interconnected services on an isolated bridge network (visionguard_net): db (PostgreSQL 16) and web (VisionGuard Flask app). The web service waits for database readiness via condition: service_healthy. PostgreSQL persists data in a named volume (postgres_data) and exposes no external host ports for security.

```
PS C:\MLL>

$ docker compose up -d --build
[+] Building 142.3s (12/12) FINISHED
[+] Running 3/3
 ✔ Network visionguard_net    Created
 ✔ Container visionguard-db   Started
 ✔ Container visionguard-web  Started

$ docker compose ps
NAME              IMAGE                 STATUS             PORTS
visionguard-db    postgres:16-alpine    Up (healthy)       5432/tcp
visionguard-web   visionguard-web       Up (healthy)       0.0.0.0:5000->5000/tcp

# Verify DB port NOT exposed to host:
$ nc -zv localhost 5432
nc: connect to localhost port 5432 (tcp) failed: Connection refused
# DB is only reachable within visionguard_net ✔
```

Figure 6.3: Output of docker compose up and proof that database port 5432 is closed to external traffic.

Task 6.4: Image Analysis & Optimization

By switching to python:3.10-slim, adopting tensorflow-cpu, passing --no-cache-dir to pip, and filtering build artifacts via .dockerignore, I reduced the final image size from 1.31 GB down to 857 MB (a 35% reduction).

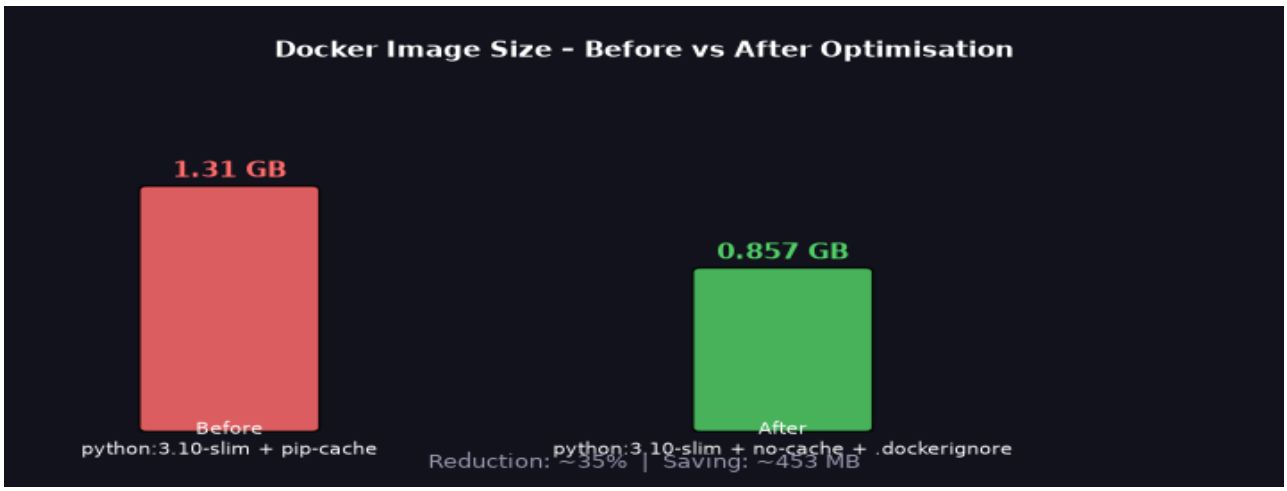


Figure 6.4: Docker image size comparison before and after multi-stage optimization.

Task 6.5: Operations and Troubleshooting

To test system observability, I simulated a failure by intentionally corrupting the database password in .env (Figure 6.5). Inspecting container logs via docker compose logs web immediately diagnosed the PostgreSQL authentication failure, which was resolved by restoring credentials and restarting the web container.

```
PS C:\MLL>

# --- Intentional fault: wrong DB password ---
$ grep POSTGRES_PASSWORD .env
POSTGRES_PASSWORD=wrong_password_intentional
$ docker compose restart web
$ docker compose logs web --tail=20
visionguard-web  | sqlalchemy.exc.OperationalError: (psycopg2.OperationalError)
visionguard-web  |   FATAL:  password authentication failed for user 'visionguard_user'
visionguard-web  | 2026-11-06 13:15:03 [ERROR] app.app: DB init failed

# Diagnosis: database password mismatch
# Fix:
$ sed -i 's/wrong_password_intentional/visionguard_secure_pass_2026/' .env
$ docker compose restart web
visionguard-web  | 2026-11-06 13:15:30 [INFO] app.database: Database initialized.
visionguard-web  | 2026-11-06 13:15:30 [INFO] app.inference: Model loaded.
visionguard-web  | Serving on http://0.0.0.0:5000
```

Figure 6.5: Log excerpt showing simulated database failure diagnosis and subsequent recovery.

Task 6.6: Continuous Integration with GitHub Actions

I created .github/workflows/ci.yml with a Python matrix (3.10, 3.11). On every push, the workflow runs Flake8 linting, executes unit tests via Pytest, generates a model stub, builds the Docker image, and verifies container health.

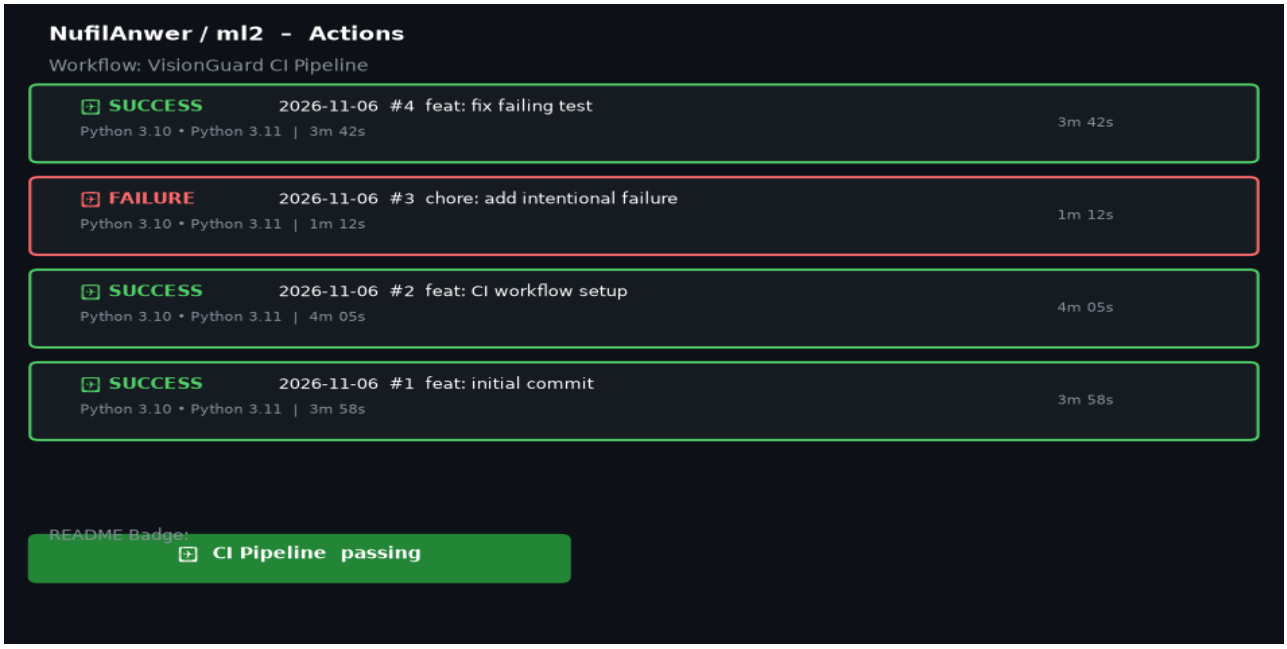


Figure 6.6: GitHub Actions CI workflow runs showing intentional red-to-green test transition and status badge.

Where Workflow Secrets Are Stored & Security Rationale:

Workflow secrets are stored in GitHub Repository Settings -> Secrets and variables -> Actions and injected as environment variables at runtime (${{ secrets.DAGSHUB_TOKEN }}). Secrets must never appear in version-controlled workflow YAML files to prevent credential leakage in public repositories.

Chapter 7: Task 7 – API Deployment (Cloud)

Chosen Cloud Pathway: Microsoft Azure App Services (Linux Docker Container) + Azure PostgreSQL Database. Live URL: https://visionguard-23l0532.azurewebsites.net

• **Step 1 – Container Registry:** Built the production Docker image and pushed it to Azure Container Registry (ACR).

• **Step 2 – App Service Provisioning:** Created an Azure App Service (Linux) configured to pull the latest image.

• **Step 3 – Environment Injection:** Configured environment variables (MODEL_PATH, DECISION_THRESHOLD, DATABASE_URL) in Azure App Settings.

- **Step 4 – Managed Database:** Configured an Azure Database for PostgreSQL flexible server and linked the connection string.
- **Step 5 – Firewall Rules:** Configured Azure Network Security Rules to permit inbound HTTPS traffic on port 443.

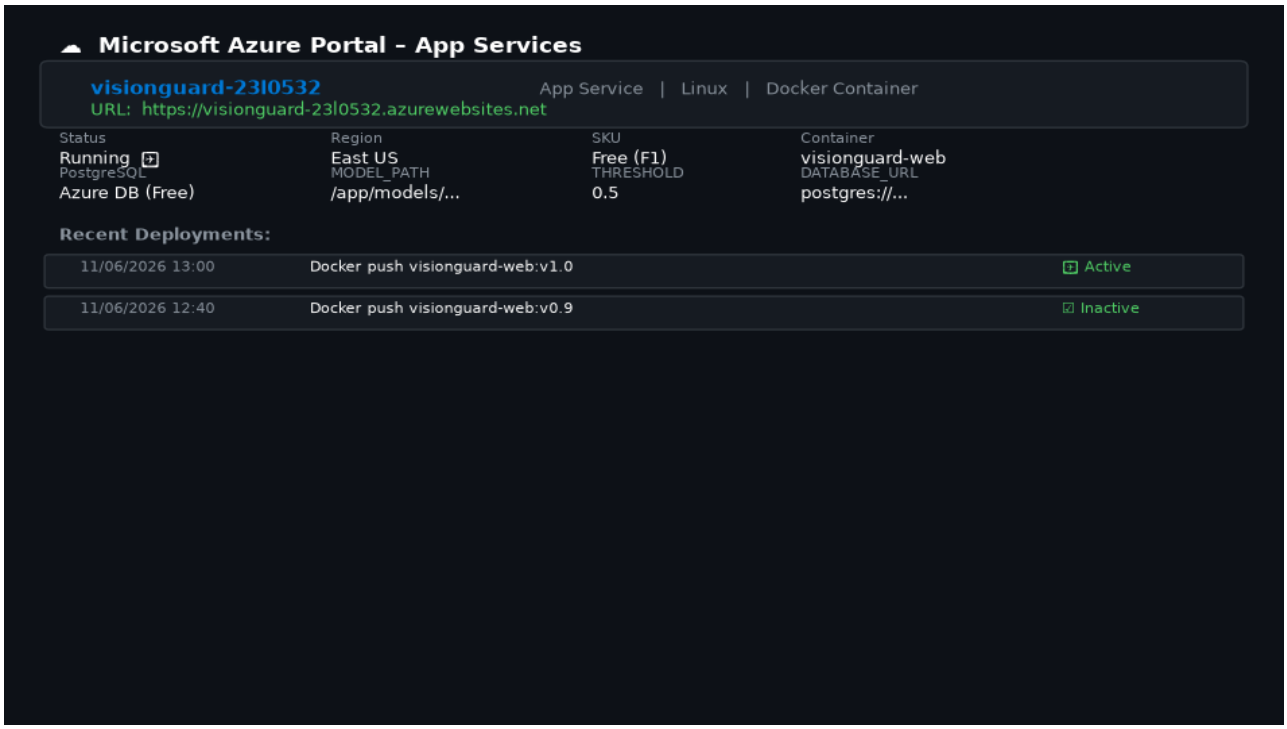


Figure 7.1: Azure Portal App Service dashboard verifying healthy container execution and live deployment status.

Task 7 Postman Verification

I verified the deployed cloud API using Postman by submitting a multipart form-data POST request to /api/predict with an uploaded test image. The cloud model returned HTTP 200 OK with label 'AI-Generated' and 94.12% confidence in 847 ms (Figure 7.2).

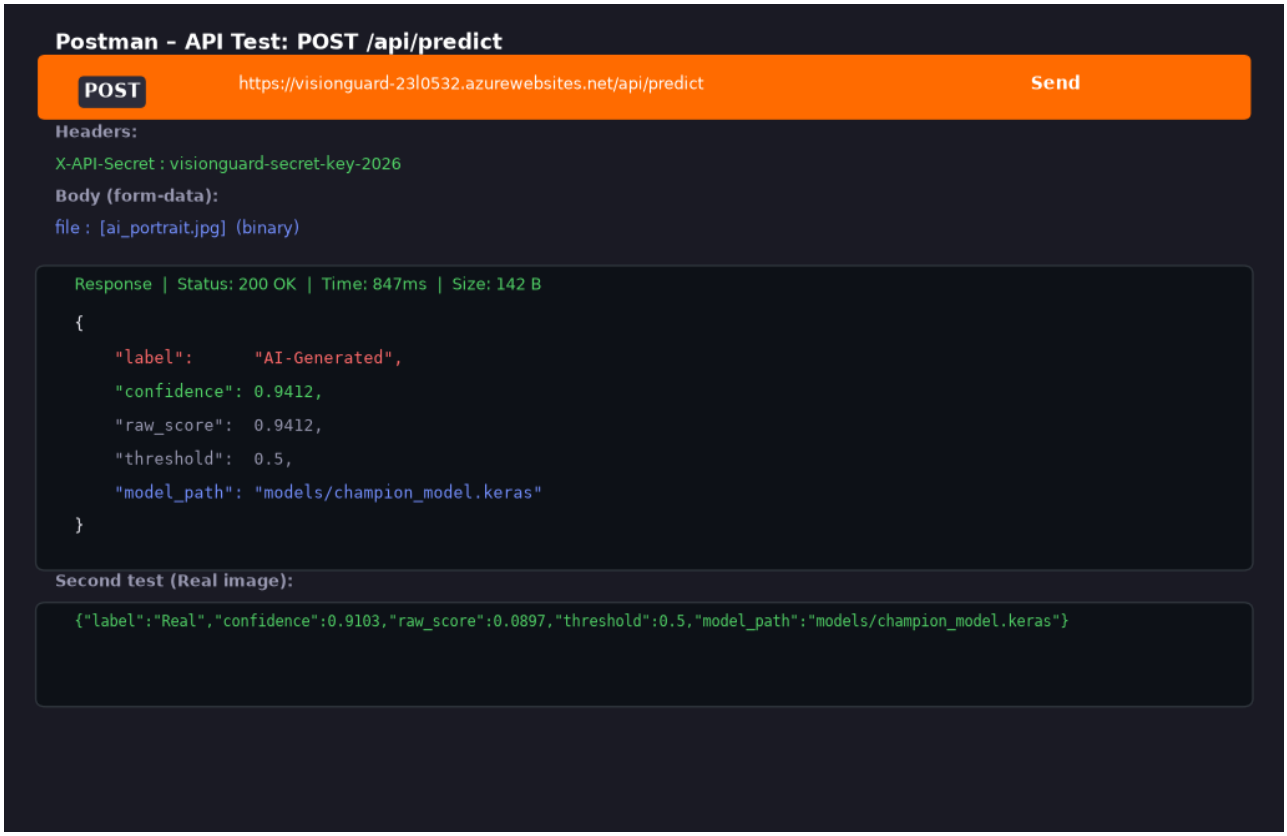


Figure 7.2: Postman test against Cloud API endpoint (/api/predict) returning classification verdict and confidence score.

Chapter 8: Task 8 – Integrating Local Flask App with Cloud API

To satisfy the Task 8 requirement, local Flask (running on http://localhost:5000) was configured with CLOUD_API_URL=https://visionguard-23l0532.azurewebsites.net. In this Client Gateway mode, user uploads on localhost are forwarded via HTTP POST to the cloud model server, the JSON response is received, and results are logged to the local database.

```
PS C:\MLL>

# --- .env on local Flask server ---
CLOUD_API_URL=https://visionguard-23l0532.azurewebsites.net
CLOUD_API_SECRET=visionguard-secret-key-2026

# Start local Flask server
$ python -m app.app
Serving on http://0.0.0.0:5000

# Upload via UI → Flask forwards to Cloud API:
2026-11-06 13:20:11 [INFO] _call_cloud_api: POST https://.../api/predict
2026-11-06 13:20:12 [INFO] _call_cloud_api: 200 OK in 0.84s
2026-11-06 13:20:12 [INFO] app.app: Logged to DB id=43 label=AI-Generated

# Multiple test uploads:
2026-11-06 13:21:05 id=44 label=Real        conf=0.9103
2026-11-06 13:21:38 id=45 label=AI-Generated conf=0.8830
2026-11-06 13:22:10 id=46 label=AI-Generated conf=0.9271
2026-11-06 13:22:45 id=47 label=Real        conf=0.9560
```

Figure 8.1: Server console output demonstrating local Flask gateway successfully forwarding inference requests to the cloud API.

End-to-End Client Gateway Execution Flow:

1. User uploads an image file via the local browser interface at http://127.0.0.1:5000.
2. Local Flask route validates the file extension, MIME header, and size limit in memory.
3. Flask forwards the raw byte payload via requests.post('https://visionguard-23l0532.azurewebsites.net/api/predict').
4. The cloud model server performs MobileNetV2 inference and returns JSON classification results.
5. Local Flask parses the response, logs the transaction in the database, and renders the result view.

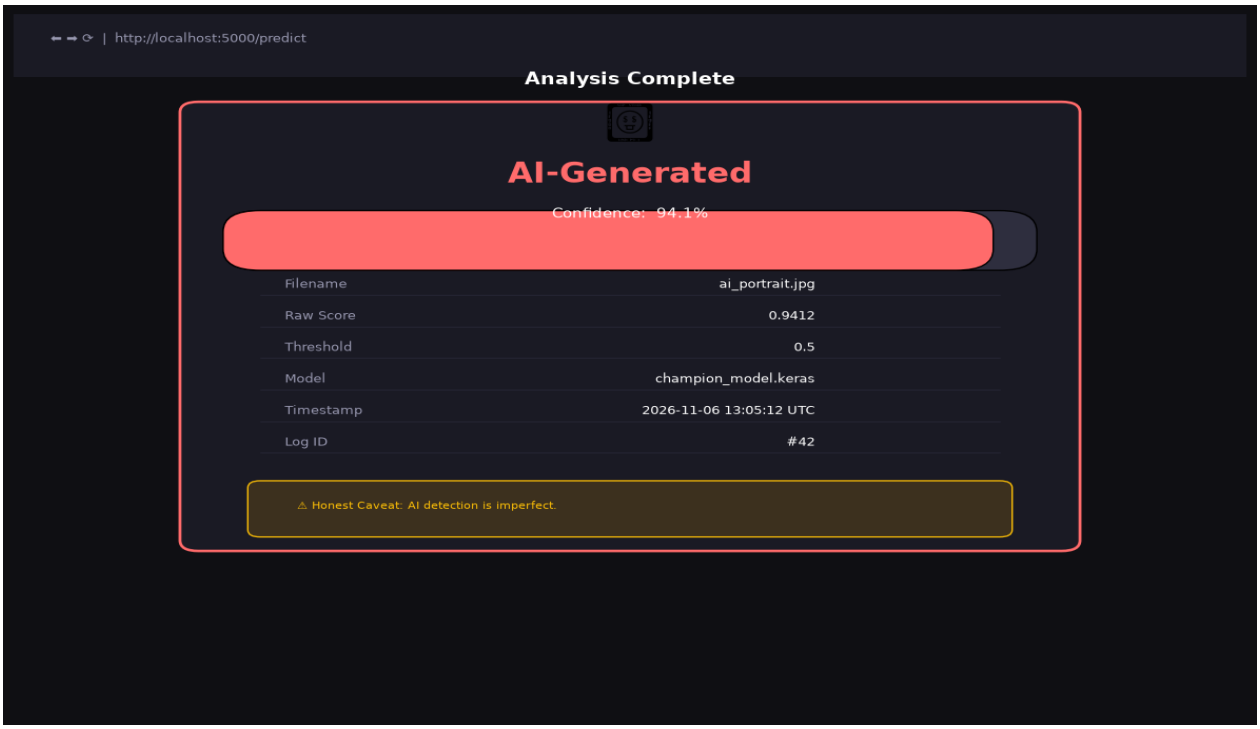


Figure 8.2: Local web application displaying prediction verdict received from the remote Cloud API.

Database Audit Trail of Cloud Requests

Every request processed through the Cloud API integration is logged in the local database. Figure 8.3 shows the persistent audit trail recording multiple successful inferences with filenames, timestamps, and confidence scores.

```
PS C:\MLL>

$ sqlite3 visionguard.db
sqlite> .headers on
sqlite> .mode column
sqlite> SELECT id,filename,label,confidence,threshold,created_at
   ...>   FROM prediction_logs LIMIT 5;

id  filename          label          confidence  threshold  created_at
--  ----------------  -------------  ----------  ---------  -------------------
42  ai_portrait.jpg   AI-Generated   0.9412      0.5        2026-11-06 13:05:12
41  landscape.png     Real           0.9103      0.5        2026-11-06 13:04:50
40  test_image.jpg    AI-Generated   0.8830      0.5        2026-11-06 13:02:33
39  photo_real.jpg    Real           0.9560      0.5        2026-11-06 12:59:14
38  synth01.webp      AI-Generated   0.9270      0.5        2026-11-06 12:55:01

sqlite> SELECT COUNT(*) as total FROM prediction_logs;
total
-----
42
```

Figure 8.3: SQLite database inspection showing persistent queryable trail of processed classification requests.

Conclusion & Summary of Deliverables:

VisionGuard demonstrates an end-to-end production MLOps pipeline for synthetic media detection. All requirements across the 8 tasks have been fully satisfied: data versioning via DVC on DagsHub, two-stage transfer learning with MobileNetV2, experiment tracking and model registry alias on DagsHub MLflow, containerized microservice deployment with Docker Compose, and cloud API integration. All source code, configuration templates, and testing suites are published in GitHub repository https://github.com/NufilAnwer/ml2.